

CampusConnect: Technical Deep Dive & Interview Guide

An Authentic, Line-by-Line Engineering & Architecture Breakdown for Interview Excellence

1. Executive Summary & Campus Problem Statement

At a modern university campus with 10,000+ students, peer-to-peer commerce and lost item recovery were fundamentally broken due to **information asymmetry**, **unstructured communication channels**, and **lack of privacy**:

- **Marketplace Friction & Message Decay:** End-of-semester rushes forced students to sell mattresses, textbooks, lab equipment, and bicycles on 1,000+ member WhatsApp/Telegram groups. Posts quickly decayed under hundreds of chat messages, leading to lowballing, ghosting, lost deals (~30% efficiency loss), and privacy exposure (posting personal phone numbers and hostel room numbers publicly).
- **The Lost & Found Panic:** Losing room keys, student IDs, or earphone cases right before midterms caused intense anxiety. While finder posts appeared sporadically in chat feeds, owners rarely saw them in time. Simple keyword searches failed due to description mismatches (e.g., owner searches for '*black hydroflask*' while finder posts '*water bottle in hall B*').

The Engineering Solution: CampusConnect was engineered as a centralized campus orchestrator — combining an anonymous, privacy-preserving marketplace with a **hybrid AI/Computer Vision Lost & Found smart-matching engine**.

2. Project Overview (STAR Framework)

Scenario A: Smart Lost & Found Matching Engine

S (Situation): Lost-and-found recovery on campus suffered from an 80% failure rate in instant chats because item descriptions were non-standardized and keyword search failed when owners and finders used different terminology.

T (Task): Engineer an automated matching engine capable of calculating similarity between new 'Lost' reports and existing 'Found' items using both textual descriptions and uploaded photos.

A (Action): Designed a hybrid pipeline combining **TF-IDF Cosine Similarity** (word frequency vector analysis) with a multi-stage **OpenCV Computer Vision pipeline** (HSV Color Histograms at 40% weight, ORB Feature Keypoints with Lowe's Ratio Test at 35% weight, and Grayscale Pixel Structural Comparison at 25% weight). Fused text scores, image scores, category bonuses (+0.15), and title substring bonuses (+0.20) into a single normalized confidence score (threshold 0.25).

R (Result): Increased match detection accuracy by ~30% over keyword-only searching, automatically recommending potential matches to students upon posting.

Scenario B: Safe Campus Marketplace & Monetization Architecture

S (Situation): Students trading high-value items faced safety risks, price gouging, and privacy exposure by revealing personal phone numbers on public group chats.

T (Task): Build a secure marketplace with identity masking, tiered fee monetization, and strict role isolation.

A (Action): Implemented an **Anonymous Handle Engine** generating unique handles (e.g., User#9X2A1) during registration, sanitized public item listings by stripping pickup locations (`del item['pickup_location']`) until purchase completion, built a **Tiered Monetization Engine** charging category-specific fees (3% Books, 12%

Electronics) and value-tiered fees for general goods, and restricted Admin accounts from trading via controller guards.

R (Result): Enabled secure campus transactions, eliminated off-platform spam, and generated predictable revenue streams through automated platform fee calculation.

3. Architecture & Complete Data Model (MongoDB Atlas)

CampusConnect follows a clean 3-tier architecture: React (Vite) frontend, Flask REST Blueprints backend (/api/auth, /api/marketplace, /api/lostandfound, /api/admin, /api/reviews), and MongoDB Atlas database cloud.

Collection	Key Fields & Types	Purpose & Security Constraints
Users	_id, email, password, role, anon_name, created_at	Primary user account. Email unique. Single admin constraint enforced at registration.
MarketplaceItems	_id, title, price, category, seller_id, seller_anon_name, status, pickup_location	Listings available/sold. pickup_location explicitly stripped in public GET payloads.
Reports	_id, user_id, user_anon_name, type, item_name, category, description, image_url, status	Lost/Found reports. Triggers smart-matching engine upon insertion.
Claims	_id, found_report_id, claimer_id, finder_id, verification_details, reward_amount, status	Claim verification flow. Admin verifies ownership before reward release.
Transactions	_id, item_id, buyer_id, seller_id, price, platform_fee, total_amount, status	Ledger recording completed purchases, base prices, and platform fee split.
Payments	_id, type, transaction_id, buyer_id, seller_id, item_price, platform_fee, status	Revenue collection records & 100% finder reward transfers.
Reviews	_id, target_user_id, reviewer_id, rating, comment, created_at	Peer ratings building seller reputation on campus.

4. Smart Matching Engine: Technical Deep Dive

When a Lost report is submitted, CampusConnect executes a multi-phase scoring pipeline comparing the target report against all Open Found reports.

Multi-Layered Scoring Algorithm Pipeline:

- TF-IDF Cosine Similarity (Text):** Tokenizes item_name and description, builds term frequency vectors using Counter, computes normalized inner product.
- Computer Vision Pipeline (OpenCV)** (if both reports have images):
 - HSV Color Histogram (40% weight):** Converts BGR to HSV, computes 2D H-S histogram (50x60 bins), compares correlation via cv2.compareHist(cv2.HISTCMP_CORREL).
 - ORB Feature Matching (35% weight):** Detects keypoints via cv2.ORB_create(nfeatures=500), matches descriptors via BFMatcher, filters via Lowe's Ratio Test (m.distance < 0.75 * n.distance).
 - Structural Grayscale Comparison (25% weight):** Resizes to 128x128 grayscale, computes absolute pixel difference cv2.absdiff, normalizes similarity score.
- Heuristic Domain Bonuses:** +0.15 for exact category match, +0.20 for title substring match.

4. **Dynamic Fusion Score:** Combined = (Text * 0.40) + (Image * 0.35) + Bonuses (or Text 70% + Bonuses 30% if no photos). Threshold ≥ 0.25 flags a smart match.

5. Security, Anonymity & RBAC Architecture

- **Anonymous Identity Engine:** During registration, the backend assigns a randomized, non-identifying handle: User# + random 5-char uppercase string (e.g. User#9X2A1). Real names and emails are never exposed on public item pages.
- **Pickup Location Obfuscation:** Pickup locations (e.g. 'Hostel 4, Room 302') are explicitly scrubbed from public marketplace API responses (`del item['pickup_location']`) until payment confirmation.
- **Controller-Level Admin Isolation:** Admins manage disputes and view transaction analytics, but are explicitly blocked at the controller layer from posting items, buying items, or claiming items.

6. Monetization & Platform Fee Rules

The backend computes dynamic platform fees using `calculate_marketplace_fee(price, category)`:

```
def calculate_marketplace_fee(price, category):
    cat_lower = str(category).lower()
    if 'book' in cat_lower:
        return round(price * 0.03, 2)      # 3% fee for books
    elif 'electronic' in cat_lower:
        return round(price * 0.12, 2)      # 12% fee for high-value electronics
    if price <= 2000:
        return round(price * 0.05, 2)      # 5% fee for items <= Rs. 2,000
    elif price <= 5000:
        return round(price * 0.08, 2)      # 8% fee for items Rs. 2,001 - 5,000
    else:
        return round(price * 0.10, 2)      # 10% fee for items > Rs. 5,000
```

7. Specific Engineering Bugs Fixed (Real Code Analysis)

Bug 1: Flask jsonify() Crash on MongoDB ObjectId & datetime Types

PyMongo query results contain native BSON ObjectId and datetime instances. Calling Flask's default jsonify() threw a `TypeError: Object of type ObjectId is not JSON serializable`. Solved globally by extending Flask's DefaultJSONProvider:

```
class CustomJSONProvider(DefaultJSONProvider):
    def default(self, obj):
        if isinstance(obj, ObjectId):
            return str(obj)
        if isinstance(obj, datetime.datetime):
            return obj.isoformat()
        return super().default(obj)

# Applied globally in app.py
app.json = CustomJSONProvider(app)
```

Bug 2: Single Admin Enforcement Constraint

Prevented rogue admin accounts by checking regex role: `^admin$` during registration. If an admin exists, subsequent admin registrations are blocked with HTTP 400.

Bug 3: Prevention of Self-Transactions & Self-Claims

Enforced `str(item['seller_id']) != buyer_id` in marketplace /buy controller and `str(report['user_id']) != claimer_id` in lostandfound /claim controller.

8. How Would You Scale It? (100,000+ Users Across 50+ Campuses)

- **CLIP Deep Learning Embeddings + Vector Search:** Replace in-memory OpenCV and TF-IDF loops with OpenAI CLIP multimodal embeddings stored in MongoDB Atlas Vector Search or Qdrant for sub-10ms k-NN similarity lookups.
- **Asynchronous Task Queue:** Offload heavy image comparison to background task queues using Celery + Redis or AWS SQS + Lambda.
- **Database Sharding & Caching:** Shard MongoDB collections by campus_id and use Redis for fast lookup of active marketplace listings.
- **Cloud Storage CDN:** Migrate image uploads from local disk to AWS S3 / Cloudinary backed by AWS CloudFront CDN.

9. Comprehensive Technical Interview Question & Answer Bank

Q1: Why did you choose MongoDB over a Relational Database like PostgreSQL?

Campus marketplace listings and lost-and-found reports have highly dynamic schemas. For instance, a lost laptop report requires fields for brand, serial number, and charger presence, while a lost textbook needs ISBN and edition details. MongoDB's document model allowed us to store unstructured metadata naturally without performing expensive ALTER TABLE migrations.

Q2: What is the main drawback of your in-memory OpenCV matching algorithm, and how would you address it in production?

The main drawback is $O(N)$ CPU compute cost per incoming report. When a user posts a report, the application thread downloads images, resizes them, and runs matrix multiplications synchronously, blocking the HTTP response thread. In production, I would offload image matching to an async task queue (Celery + Redis) and migrate from pixel-level OpenCV comparison to pre-computed CLIP neural vector embeddings indexed in a vector database for $O(\log N)$ search time.

Q3: How do you handle authentication securely across stateless REST endpoints?

We use stateless JSON Web Tokens (JWT) via Flask-JWT-Extended. Upon successful password verification (hashed via `werkzeug.security.generate_password_hash` using PBKDF2), the server generates a signed JWT containing the user's `_id` identity and custom role claim. The client transmits this token in the Authorization: Bearer header for subsequent requests.

Q4: How do you protect student privacy while facilitating peer-to-peer campus trade?

We implement Privacy by Design: (1) Every user is assigned a random handle (User#9X2A1) at registration; real names and emails are never exposed on public item pages. (2) Pickup locations are scrubbed from public marketplace API responses until a valid payment is confirmed. (3) Buyer and seller identities are masked during browsing.

Q5: What happens if an image URL in Lost & Found fails to download during similarity computation?

The image similarity module wraps network requests in try-except blocks with a 10-second timeout. If image downloading fails or OpenCV throws an exception, the system gracefully catches the error, sets the image score to 0.0, and falls back to a text-only TF-IDF scoring model (70% text score + 30% category/name bonuses).

Q6: Explain Lowe's Ratio Test used in your ORB feature matching.

ORB detects keypoints and generates binary descriptors. When matching descriptors between two images using K-Nearest Neighbors ($K=2$), we obtain the two closest descriptor matches (m and n). Lowe's Ratio Test compares their distances: if $m.distance < 0.75 * n.distance$, it signifies that the top match is significantly closer than the second-best match, filtering out ambiguous or noisy keypoints.

Q7: How did you implement Role-Based Access Control (RBAC)?

RBAC is implemented at both token generation and controller levels. JWT tokens carry embedded role claims (Student or Admin). Custom decorators like `@admin_required` verify `claims.get('role') == 'Admin'`. Furthermore, student controllers explicitly check `claims.get('role')` and reject Admin accounts attempting student-only actions with HTTP 403 Forbidden.

Q8: Why use HSV color space instead of RGB for color histogram matching?

RGB represents colors as combinations of Red, Green, and Blue, making color intensity heavily sensitive to lighting and shadow variations. HSV (Hue, Saturation, Value) separates color tone (Hue) and intensity (Saturation) from brightness (Value). By analyzing only the H and S channels, our histogram matching remains robust against variations in ambient lighting.

10. Interview Summary Cheat Sheet

Topic	Concise Answer	Key Code / Spec
Core Value	Centralized campus commerce & AI-assisted lost item recovery.	Eliminates WhatsApp group chat chaos.
Tech Stack	React (Vite), Flask, MongoDB Atlas, Stripe API, OpenCV, JWT.	Flask Blueprints architecture.
Smart Matching	Hybrid TF-IDF Cosine Text + OpenCV Vision (HSV, ORB, SSIM).	Score threshold ≥ 0.25 triggers auto-match.
Image CV Weights	HSV Color (40%), ORB Features (35%), Structural SSIM (25%).	Lowe's ratio test threshold 0.75.
Anonymity Engine	Random handle (User#9X2A1) generated during registration.	Hides student name & email publicly.
Location Protection	Pickup location removed from public GET /items payload.	<code>del item['pickup_location']</code> in route.
Key Architecture Fix	Custom Flask JSON Provider for clean MongoDB ObjectId serialization.	Extends DefaultJSONProvider globally.
Monetization Model	Tiered marketplace fees (Books 3%, Electronics 12%, General 5-10%).	100% of rewards go to finder.
Admin Governance	Full dispute oversight; controller-blocked from personal trading.	Enforces <code>@admin_required</code> & 403 blocks.
Scaling Strategy	CLIP Neural Embeddings + Vector Search (k-NN) + Celery/Redis.	Replaces in-memory loop for 100k+ scale.